



The Art of Writing Efficient Programs

1. Performance Fundamentals

▼ Introduction to Performance and Concurrency

Efficiency deals with using resources optimally and not wasting them. An efficient program makes good use of the computational hardware. On the one hand, an efficient program does not leave available resources idle: if you have a computation that needs to be done and a processor that is not doing anything, that processor should be executing the code that is waiting to be executed. The idea goes deeper: processors have many computing resources in them, and an efficient program tries to make use of as many of these resources as possible at the same time. On the other hand, an efficient program does not waste resources doing unnecessary work: it does not perform computations that do not need to be done, does not waste memory to store data that is never going to be used, does not send data over the network if it's not needed, and so on. In short, an efficient program does not leave the available hardware idle and does not do any work that doesn't have to be done.

Performance, on the other hand, always relates to some metrics. The most common one

is "speed," or how fast the program is. The more rigorous way to define this metric is the **throughput, which is the amount of computations the program does in a given time.** The inverse metric that is often used for the same purpose is the turnaround time or how much time is needed to compute a particular result. However, this is not the only possible definition of performance.

▼ Performance Measurements

Core principle: Never guess about performance. Every performance-related decision should be driven by measurements and data.

1. Why Performance Measurement Matters

When optimizing a program, we need to answer three questions:

1. **How fast is the program currently?**
2. **Where is the program spending its time?**
3. **Did our optimization actually improve performance?**

Performance is not intuitive. Code that looks faster can actually be slower because of compiler optimizations, CPU architecture, memory behavior, etc.

Therefore:

Measure → Analyze → Optimize → Measure again

Performance measurements are required throughout the entire optimization process, not just at the beginning and end.

2. Three Main Performance Measurement Techniques

- a. Benchmarking: Measures **how long the program/code takes.**
- b. Profiling: Tells you **where the program spends its time.**
- c. Micro-benchmarking: Measures **small pieces of code in isolation.**

3. Performance profiling

There are several different types of profilers:

- a. Some profilers execute the code under an interpreter or a virtual machine and observe where it spends the time. The main downside of these profilers is that they make the program run much slower than the code compiled directly to machine instructions, at least for languages like C
- b. Other profilers require that the code is instrumented with special instructions during compilation or linking. These instructions provide additional information to the profiler, for example, so that they can notify the data collection engine when a function is called or a loop begins and ends. These profilers are faster than the ones of the previous type but still slower than the native execution. They also require a special compilation of the code and rely on the assumption that the instrumented code has the same performance as the original code, at least relatively, if not absolutely.
- c. **Most modern profilers use the hardware event counters that are present on all modern CPUs.** These are special hardware registers that can be used to track certain hardware events. An example of a hardware event is executing an instruction. You can see how this can be useful for profiling: the processor will do the work of counting instructions for us without any additional instrumentation or any overhead. All we need to do is to read the values of the counter registers.

4. The perf profiler

- a. This profiler uses hardware performance counters and time-based sampling; it does not require any instrumentation of the code. The simplest way to run this profiler is to collect the counter values for the entire program; this is done using the perf stat command:

```
$ clang++-11 -O3 -mavx2 -Wall -pedantic compare.C example.C -o example
$ perf stat ./example
Sort time: 156ms (276557 comparisons)

Performance counter stats for './example':

      158.048821      task-clock (msec)          #    0.997 CPUs utilized
           2          context-switches          #    0.013 K/sec
           0          cpu-migrations            #    0.000 K/sec
          209         page-faults              #    0.001 M/sec
    497,045,599       cycles                   #    3.145 GHz
    1,355,549,089     instructions              #    2.73   insn per cycle
    450,694,541       branches                 # 2851.616 M/sec
      389,020        branch-misses             #    0.09% of all branches

      0.158582626 seconds time elapsed
```

- b. As it turns out, modern CPUs can collect statistics on many different types of events, but only a few types at a time; in the preceding example, eight counters were reported, so we can assume that this CPU has eight independent counters. However, each of these counters can be assigned to count one of many event types. The profiler itself can list all the events that are known to it and can be counted:

```
$ perf list

List of pre-defined events (to be used in -e):

branch-instructions OR branches      [Hardware event]
branch-misses                        [Hardware event]
bus-cycles                           [Hardware event]
cache-misses                         [Hardware event]
cache-references                     [Hardware event]
cpu-cycles OR cycles                 [Hardware event]
instructions                         [Hardware event]
ref-cycles                           [Hardware event]
```

- c. If you want to collect other counters for profiling:

```
$ perf stat -e cycles,instructions,branches,branch-misses,cache-references,cache-misses ./example
Sort time: 109ms (276557 comparisons)

Performance counter stats for './example':

    342,547,009       cycles                   (63.98%)
    1,333,447,617     instructions              (82.09%)
    448,700,032       branches                 (85.52%)
     443,370         branch-misses             (85.51%)
    1,555,766         cache-references          (85.51%)
     168,003         cache-misses             (79.47%)
#    0.10% of all branches
# 10.799 % of all cache refs

      0.111470330 seconds time elapsed
```

- The instruction counter measures the number of processor instructions that were executed; as you can see, the CPU

executes, on average, almost four instructions per cycle.

- The "branches" are the conditional instructions: every if statement and every for loop with a condition generates at least one of these instructions.
 - The "cache references" count how many times the CPU needed to fetch something from memory.
- d. The cycle time is the inverse of the CPU frequency, so a 3GHz CPU can run 3 billion cycles per second. By the way, most CPUs can run at variable speeds, which complicates the measurements.

5. Detailed profiling with perf

- a. The perf profiler combines the hardware counters with time interval-based sampling to record the profile of the running program. For each sample, it records the position of the program counter (the address of the instruction to be executed) and the values of the performance counters that we are monitoring. After the run, the data is analyzed; the functions and code lines with the most samples are responsible for the majority of the execution time.

```
-----  
$ clang++-11 -g -O3 -mavx2 -Wall -pedantic compare.C example.C -o example  
$ perf record ./example  
Sort time: 107ms (276557 comparisons)  
[ perf record: Woken up 1 times to write data ]  
[ perf record: Captured and wrote 0.037 MB perf.data (419 samples) ]
```

- b. Just like perf stat, we could have specified a counter or a set of counters to monitor but, this time, we accept the default counter. We haven't specified how often the samples are taken; again, there is a default for that, but we could also specify it explicitly: for example, `perf record -c 1000` records 1000 samples per second.
- c. The program runs, produces its regular output, as well as the messages from the profiler. The last one tells us that the profiling samples have been captured in the file named perf.data (again, this is the default that can be changed). To visualize the data from this file, we need to use the profile analysis tool, which is also a part of the same perftools suite, specifically, the `perf report` command. Running this command will launch this screen:

Samples: 453 of event 'cycles:ppp', Event count (approx.): 362699054

Overhead	Command	Shared Object	Symbol
96.46%	example	example	[.] compare
1.39%	example	example	[.] std::__introsort_loop<__gnu_cxx::
0.64%	example	example	[.] main
0.59%	example	[kernel.kallsyms]	[k] vma_interval_tree_insert
0.56%	example	[kernel.kallsyms]	[k] filemap_map_pages
0.21%	example	[kernel.kallsyms]	[k] _raw_spin_lock_irqsave
0.14%	example	[kernel.kallsyms]	[k] perf_event_mmap_output
0.01%	perf	[kernel.kallsyms]	[k] __x86_indirect_thunk_r14
0.00%	perf	[kernel.kallsyms]	[k] native_apic_mem_write
0.00%	perf	[kernel.kallsyms]	[k] native_write_msr

6. The Google Performance profiler

a.